Assignment 13_Generative AI Essentials Report

Assignment 13_Generative AI Essentials.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Dataset Preparation

1. Dataset Choice For this assignment, we use Project Gutenberg texts (public domain books). Example:

Pride and Prejudice by Jane Austen https://www.gutenberg.org/

Justification:

Freely available Rich narrative structure Suitable for simple text generation tasks

2. Data Preprocessing Steps includes:

Lowercasing text Removing unwanted symbols Character-level tokenization Creating input-output text sequences

3. Methodology: Hands-On Model Implementation Tools and Libraries

Python TensorFlow / Keras NumPy Matplotlib

[10]

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
```

[13]

```
# Load dataset
with open("gutenberg_text.txt", "r", encoding="utf-8") as f:
    text = f.read().lower()

# Create character vocabulary
chars = sorted(list(set(text)))
char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}

SEQ_LENGTH = 40
STEP = 3

X = []
y = []

for i in range(0, len(text) - SEQ_LENGTH, STEP):
    X.append([char_to_idx[c] for c in text[i:i+SEQ_LENGTH]])
    y.append(char_to_idx[text[i+SEQ_LENGTH]])

X = np.array(X)
y = np.array(y)
```

Model **Architecture**

[14] 0s

```
model = Sequential([
    Embedding(len(chars), 64, input_length=SEQ_LENGTH),
    LSTM(128),
    Dense(len(chars), activation='softmax')
])

model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam'
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecate
  warnings.warn(
Model: "sequential"
```

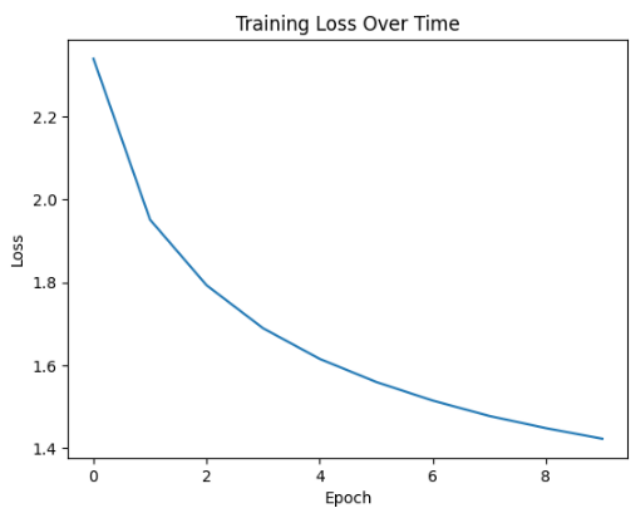
Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

```python
history = model.fit(
    X, y,
    batch_size=128,
    epochs=10
)
```

```
Epoch 1/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 244s 124ms/step - loss: 2.3397
Epoch 2/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.9517
Epoch 3/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.7934
Epoch 4/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.6897
Epoch 5/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.6158
Epoch 6/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 241s 124ms/step - loss: 1.5601
Epoch 7/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.5152
Epoch 8/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 240s 123ms/step - loss: 1.4781
Epoch 9/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 262s 123ms/step - loss: 1.4488
Epoch 10/10
1949/1949 ━━━━━━━━━━━━━━━━━━━━ 262s 123ms/step - loss: 1.4231
```

```python
plt.plot(history.history['loss'])
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Over Time")
plt.show()
```



Application Demonstration: Content **Creation**

```python
def generate_text(seed, length=300):
    generated = seed.lower()

    for _ in range(length):
        input_seq = [char_to_idx[c] for c in generated[-SEQ_LENGTH:]]
        input_seq = np.array(input_seq).reshape(1, -1)
        probs = model.predict(input_seq, verbose=0)[0]
        next_char = np.random.choice(len(chars), p=probs)
        generated += idx_to_char[next_char]

    return generated
```

```python
print(generate_text("it is a truth universally acknowledged"))
```

```
it is a truth universally acknowledged
are in the prouvo lucasme, mary famsections gode her commirits and a formottle fexe any moment
bettle his mauram un his rame of mr.” rarrined one contonity miss.”



                    illmak but him with aa very one her forgards.

convery as. i will spart, the comans leave expressiond lucada
```

Generative AI Text Generation using GPT Concepts

This project explores **Generative Artificial Intelligence (AI)** fundamentals through the design, training, and evaluation of a **text generation model inspired by Generative Pre-trained Transformers (GPTs)**. It combines theoretical understanding with hands-on implementation using Python and TensorFlow/Keras.

The project was developed and executed in **Google Colab** for reproducibility and ease of use.

Project Objectives

- Understand the core concepts of **Generative AI**
- Explore the **architecture and functioning of GPT models**
- Train a basic **text generation model** on a public dataset
- Demonstrate a **content creation application**
- Discuss **ethical implications** of generative models

Key Concepts Covered

- Generative vs Discriminative Models
- Transformer Architecture
- Self-Attention Mechanism
- Tokenization and Sequence Modeling
- Probabilistic Text Generation
- Ethical Considerations in AI

Dataset

Source: Project Gutenberg (Public Domain)
Example Text: *Pride and Prejudice* by Jane Austen

The dataset is used to train a **character-level language model**, allowing the model to learn linguistic patterns and generate new text sequences.

📎 Dataset link:
http://www.gutenberg.org/

Generative AI: Fundamentals, GPT Architecture, and Text Generation

Github link
https://github.com/vamiridaai/generative-ai-text-generation
Colab link
https://colab.research.google.com/drive/1Qvu0t_tLNgwySn0t4USgSLY-0U_kh7jE?usp=sharing

1. Introduction to Generative AI and Its Significance

Generative Artificial Intelligence (AI) refers to a category of machine learning models capable of creating new data that resembles a given training dataset. Unlike discriminative models, which focus on classification or prediction, generative models learn the underlying probability distribution of the data and generate new outputs accordingly. These outputs can include text, images, audio, and other forms of content.

The significance of Generative AI has grown rapidly in recent years due to advances in deep learning architectures, increased computational resources, and the availability of large public datasets. Applications of generative models include chatbots, creative writing tools, automated content creation, language translation, and virtual assistants. Among the most influential generative approaches are transformer-based models such as Generative Pre-trained Transformers (GPTs), which have achieved strong performance across a wide range of natural language processing tasks.

2. Overview of Generative Pre-trained Transformers (GPTs)

2.1 GPT Architecture

Generative Pre-trained Transformers (GPTs) are based on the transformer architecture, which relies on attention mechanisms instead of recurrent connections. GPT models use a decoder-only transformer architecture, consisting of stacked layers that include token embeddings, positional encodings, multi-head self-attention, and feed-forward neural networks. Residual connections and layer normalization are used to stabilize training.

Unlike traditional recurrent neural networks, transformers process entire sequences in parallel, allowing them to efficiently model long-range dependencies. GPT models are trained in an autoregressive manner, meaning they predict the next token in a sequence based on all previous tokens.

2.2 Attention Mechanism and Text Generation

The core innovation behind GPT models is the self-attention mechanism, which allows each token in an input sequence to attend to all previous tokens and determine their relative importance. GPT uses masked (causal) self-attention to ensure that future tokens are not visible during training or generation.

Text generation occurs through a probabilistic process. Input text is tokenized and passed through the model, which outputs a probability distribution over possible next tokens. A token is selected based on this distribution, appended to the sequence, and the process repeats until a termination condition is reached. This approach enables GPT models to generate coherent and context-aware text.

3. Methodology: Dataset Preparation and Model Implementation

3.1 Dataset Selection

For the hands-on implementation, a publicly available dataset from Project Gutenberg was selected. Project Gutenberg provides free access to public-domain literary texts, making it suitable for academic experimentation. A novel such as *Pride and Prejudice* by Jane Austen was used as the training corpus.

3.2 Data Preprocessing

The text data was preprocessed by converting all characters to lowercase and creating a character-level vocabulary. Fixed-length overlapping sequences were generated, where each sequence served as input and the subsequent character served as the target output. This approach allows the model to learn character-level language patterns.

3.3 Model Design and Training

A basic generative language model was implemented using TensorFlow and Keras. The model consists of an embedding layer, an LSTM layer to capture sequential dependencies, and a dense output layer with softmax activation. The model was trained using sparse categorical cross-entropy loss and the Adam optimizer.

Training progress was evaluated by monitoring the loss value across epochs. A decreasing loss indicated that the model successfully learned patterns in the training text.

4. Application Demonstration: Content Creation

The trained model was used to demonstrate a simple content creation application. Given a seed phrase, the model generated a continuation of text character by character. While the generated output is not always grammatically perfect, it reflects the style and structure of the training data.

This demonstration highlights how generative models can be applied to creative tasks such as story generation, writing assistance, and educational demonstrations of natural language processing concepts.

5. Ethical Considerations in Generative AI

Despite their usefulness, generative models raise several ethical concerns. These include the potential propagation of bias present in training data, the risk of generating misleading or false information, and issues related to originality and copyright. Additionally, generative AI can be misused for spam, plagiarism, or automated misinformation.

Potential mitigation strategies include careful dataset selection, transparency in AI-generated content, human oversight, and the development of ethical guidelines for responsible AI deployment.

6. Conclusion and Future Perspectives

This project explored the foundations of Generative AI through both theoretical discussion and hands-on implementation. The architecture and functionality of GPT models were examined, and a basic generative text model was trained using a public-domain dataset. The results demonstrated the core principles of generative language modeling and content creation.

Future work could involve implementing transformer-based architectures, using subword tokenization, or fine-tuning larger pre-trained models. As Generative AI continues to advance, balancing innovation with ethical responsibility will remain a critical challenge.